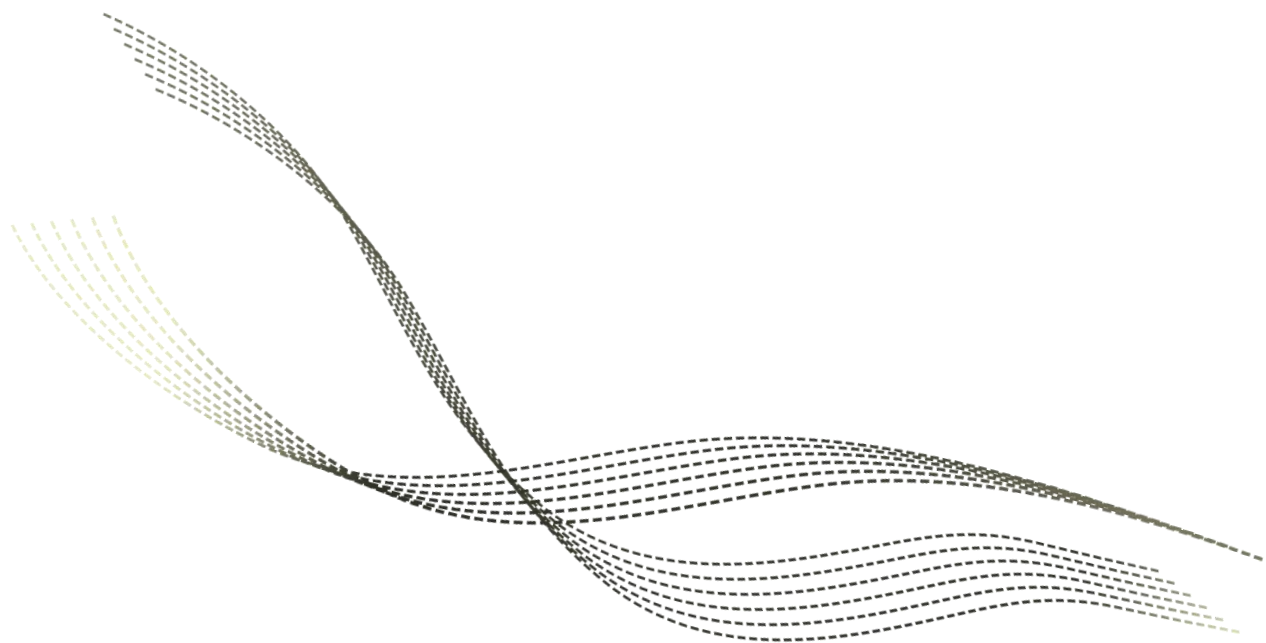


超大规模集成电路设计导论

Very Large Scale Integration Circuit (VLSI) Design

集成电路划分方法

沈明华



目录

CONTENTS

01

电路划分

02

贪心算法

03

搜索算法

04

分析算法

PART 03

搜索算法

■ 搜索算法

□ 问题建模

- 组合优化问题(例如最优划分)建模成状态空间搜索问题
- 状态表示一种可能的组合方式
- 状态空间是所有可能的组合构成的集合
- 存在邻居结构, 即一种组合可以在一步内转换成另一种组合
- 存在代价函数, 可评估所有的状态
- 寻找一种状态使得代价函数最小化(或最大化)

■ 搜索算法

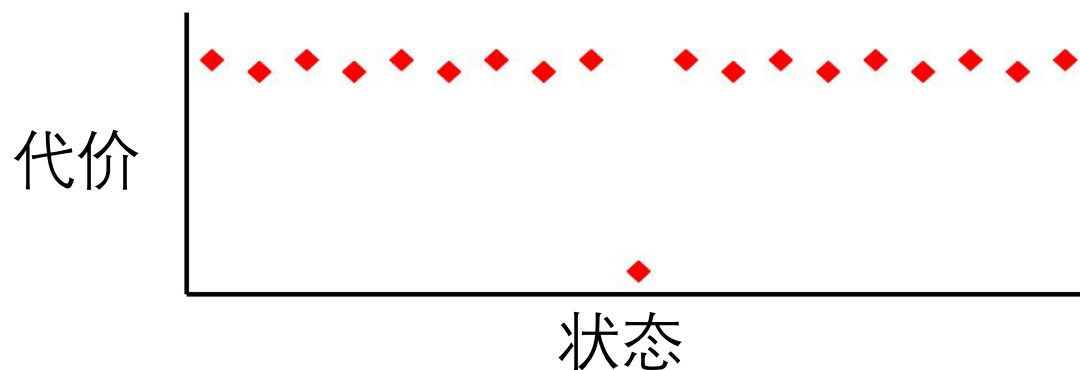
□ 贪心策略

- 是一种非常简单状态空间搜索算法
- 可以从任意状态出发
- 总是移动到最优的邻居状态
- 在周围邻居状态都比当前位置更差时，算法停止

■ 搜索算法

□ 贪心算法的不足之处

- 容易陷入局部最小值
- 会搜索到不是全局最优的解，例如下图就很难搜索到最优



- 只有凸函数或凹函数才能搜索到最优解



■ 模拟退火算法

□ 模拟退火

- 一种具有普适性的搜索算法
- 通过概率性移动避免陷入局部最小值
- 以下论文首次提出模拟退火这种启发式优化方法
 - Kirkpatrick, Gelatt and Vecchi, “Optimization by Simulated Annealing” , Science, 220(4598):498-516, May 1983.

■ 模拟退火算法

□ 基本思想

- 从退火过程获得灵感
 - 小心冷却金属材料以获得较好晶体结构
 - 首先，金属材料被加热到极高温度
 - 然后缓慢冷却金属
 - 通过适当冷却速率，原子有更大几率恢复到合适的晶体结构
- 在模拟退火中找最优状态，类似于金属退火得到最优晶体结构

■ 模拟退火算法

□ 算法流程

Algorithm 3: 模拟退火划分算法

Data: 图 G , 顶点集合 V 和普通边集合 E ,
初始温度 T 和结束温度 T_0 , 内循环迭代次数 n

Result: 割代价最小的一种划分 X, X'

```
1 随机初始化一个原始解  $X_0 \leftarrow (X, X')$ ;  
2 计算划分  $X_0$  的割代价  $c(X_0)$ ;  
3  $t \leftarrow T, X_{best} \leftarrow X_0$ ;  
4 while  $t > T_0$  do  
5   for  $i = 1$  to  $n$  do  
6     生成一个新解  $X_{new} \leftarrow generate(X_k)$ ;  
7     计算割代价  $c(X_{new})$ ;  
8     if  $c(X_{new}) < c(X_k)$  then  
9        $X_k \leftarrow X_{new}$ ;  
10      if  $c(X_k) < c(X_{best})$  then  
11         $X_{best} \leftarrow X_k$ ;  
12      continue;  
13      if  $random(0,1) < e^{-[c(X_{new})-c(X_k)]/t}$  then  
14         $X_k \leftarrow X_{new}$ ;  
15  降温  $t \leftarrow drop(t)$ ;
```

Output: 最优划分 X_{best}

■ 模拟退火算法

□ 模拟退火算法需要确定的内容

- 状态空间——所有可能的划分方案
- 相邻状态的定义——可以经过简单步骤变形而来的划分方案
- 代价函数——割代价
- 初始状态
- 初始温度
- 冷却策略(如何改变温度 t)
- 结束温度(何时退出搜索)

■ 模拟退火算法

□ 初始温度、冷却策略和结束温度通常通过实验设定

– 例如以下常见的冷却策略

➤ $t = \alpha t$, 其中 α 通常是0.95

➤ $t = e^{-\beta t} t$, 其中 β 通常是0.7

➤

■ 模拟退火算法

□ Johnson, Aragon, McGeoch and Schevon^[1]使用模拟退火解决二分划分问题

- 迭代优化方法与模拟退火方法的对比研究
- 结论：模拟退火方法是一个有效的方法，在随机图上能搜到更优解

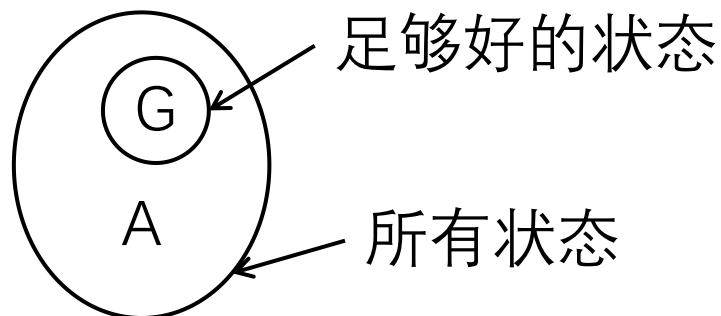
– 注意

- 模拟退火方法非常慢，所以Kernighan-Lin和Fiduccia-Mattheyses方法仍受欢迎
- 用不同初始状态多次跑Kernighan-Lin和Fiduccia-Mattheyses算法，有可能搜索到和模拟退火差不多的状态

[1] David S. Johnson, Cecilia R. Aragon, Lyle A. McGeoch, Catherine Schevon, "Optimization by simulated annealing: an experimental evaluation. Part I, graph partitioning", Open research, 1989.

■ 随机搜索算法

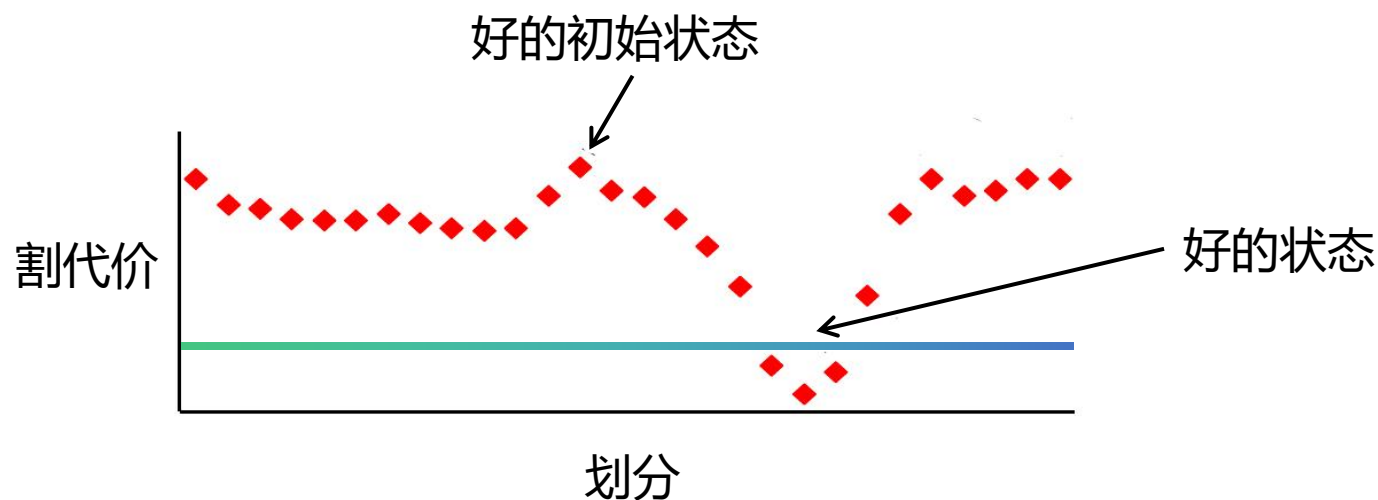
- 对于任意划分问题



- 假设状态都会随机选中
- 如果 $|G|/|A| = r$, 那么在 $5/r$ 次尝试中选到一次好状态概率 $P = 1 - (1 - r)^{5/r}$
- 例如 $|G|/|A| = 0.001$, 那么在5000次尝试中选到一次好状态的概率 $1 - (1 - 0.001)^{5000} = 0.9933$
- 核心思路: 暴力抽奖, 只要我抽的多就一定会中

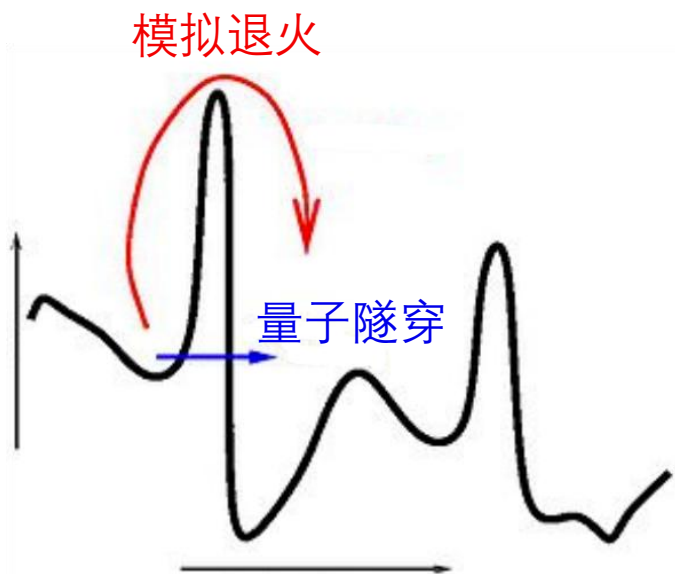
■ 随机搜索算法

- 结合Kernighan-Lin/Fiduccia-Mattheyses和随机性是一种可行策略
 - 事实上，好的状态数量十分稀少，导致 r 十分接近0
 - 如果不依赖KL/FM算法纯靠随机选取找到好状态的耗时过长
 - 因此，多次运行不同随机初始状态的KL/FM算法是一个不错的方案



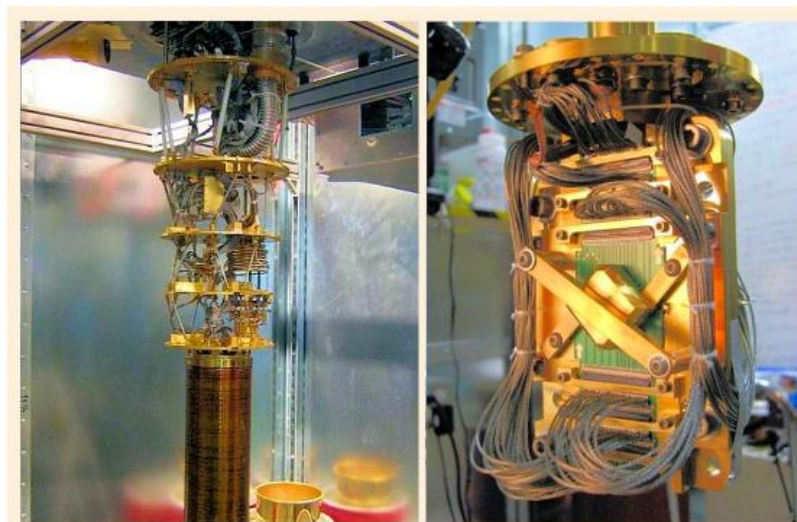
■ 随机搜索算法

□ 量子退火也是一个不错的方案



分析和数不清的证据表示，在一定条件下量子退火方法效果好于普通的模拟退火方法。

——维基百科



■ 随机搜索算法

□ 一些其他的搜索算法

- KL/FM-SA混合：使用KL/FM方法为模拟退火找到一个足够好的初始解，然后用较低温度的模拟退火方法进一步优化初始解
- 禁忌搜索算法
- 种群算法
-

PART 04

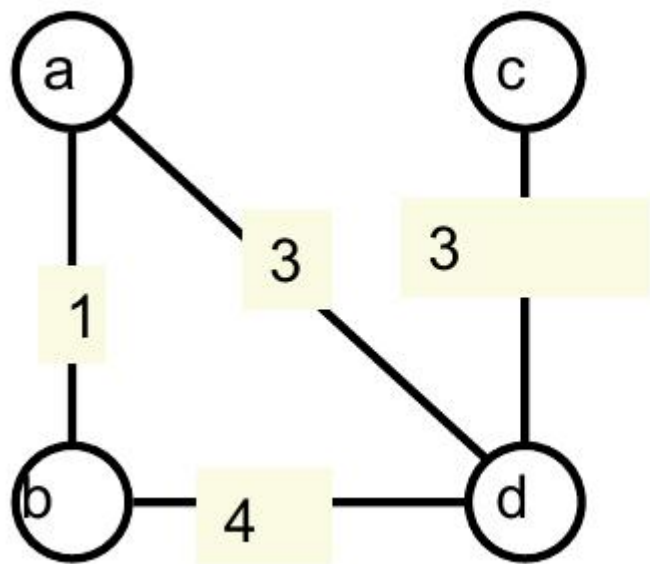
分析算法

■ 谱聚类与流分析算法

- 介绍两种优雅的划分算法
 - 尽管运行效率不是很高
- 重点是学习如何建模问题，这是VLSI CAD的关键
 - 基于谱聚类的划分算法(Spectral based partition algorithms)
 - 基于最大流的划分算法(Max-flow based partition algorithm)

■ 谱聚类算法

□ 什么是谱



$$A = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 3 \\ 1 & 0 & 0 & 4 \\ 0 & 0 & 0 & 3 \\ 3 & 4 & 3 & 0 \end{bmatrix} \end{matrix}$$

$$D = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 10 \end{bmatrix} \end{matrix}$$

A表示邻接矩阵，D表示度矩阵， $Q=D-A$ 表示拉普拉斯矩阵
谱则是由Q矩阵的特征向量构成，稍后说明如何从谱的计算得到划分

■ 谱聚类算法

□ 特征向量与特征值

- 简单回忆下矩阵中特征向量与特征值的概念
 - 如果 $Ax = \lambda x$, 其中 A 是矩阵, x 是一个向量
 - 那么 λ 是矩阵 A 的特征值, x 是矩阵 A 的特征向量
 - 注意 Kx 也是特征向量, 对于任意常数 K 都成立

■ 谱聚类算法

□ 矩阵计算的基本特点之一

$$\begin{aligned}x^T A x &= (x_1, \dots, x_n) \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{bmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \\&= \left(\sum_{i=1}^n x_i a_{i1}, \dots, \sum_{i=1}^n x_i a_{in} \right) \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \\&= \sum_{i,j} x_i x_j a_{ij}\end{aligned}$$

■ 谱聚类算法

□ 谱聚类划分算法基本方法

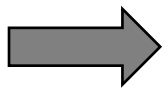
- 给定一个二等分划分集合 (X, X') , 对应一个划分向量
- $x = (x_1, x_2, \dots, x_n)$, 且 $x_i = \begin{cases} 1, & i \in X \\ -1, & i \in X' \end{cases}$
 - 很显然这个向量非零且正交于全1向量
- 令集合 $S = \{x \perp 1 \text{ and } x \neq 0\}$ 是问题的解空间
 - 找到最优划分向量使得总的割代价最小的问题可以建模为找到 $x \in S$ 使得 $\frac{1}{4} \sum_{(i,j) \in E} (x_i - x_j)^2$ 最小化
 - $\sum_{(i,j) \in E} (x_i - x_j)^2$ 从划分角度很好理解, 恰好代表了割代价。实际上是ratio-cut
 - 当 x_i 和 x_j 属于一个划分时, $(x_i - x_j)^2 = 0$
 - 当 x_i 和 x_j 属不同划分时, $(x_i - x_j)^2 > 0$

■ 谱聚类算法

□ 转化为最小化 $x^T Q x$

$$d_i = \sum_{j \neq i} A_{ij}$$

代入



$$= x^T D x - x^T A x$$

$$= \sum d_i x_i^2 - \sum A_{ij} x_i x_j$$

$$= \frac{1}{2} \left(\sum d_i x_i^2 - \sum_{i,j=1} 2A_{ij} x_i x_j + \sum d_j x_j^2 \right)$$

$$= \frac{1}{2} \left(\sum_{i=1} \left(\sum_{j=1} A_{ij} \right) x_i^2 - \sum_{i,j=1} 2A_{ij} x_i x_j + \sum_{j=1} \left(\sum_{i=1} A_{ji} \right) x_j^2 \right)$$

$$= \frac{1}{2} \sum_{i,j=1} A_{ij} (x_i - x_j)^2$$

$$= \sum_{(i,j) \in E} (x_i - x_j)^2$$

$$= 4 \times C(X, X')$$

如果是无权图可以忽略 A_{ij}
注意 $(x_i - x_j)^2 = (x_j - x_i)^2$
所以可以去掉 $1/2$



因为我们设定 ± 1 表示划分
 $(x_i - x_j)^2 = 4$ 而实际上权重只有 1

■ 谱聚类算法

□ 证明最小化 $\sum_{(i,j) \in E} (x_i - x_j)^2$ 等效于最小化ratio-cut

– 为方便计算定义 $x_i = \begin{cases} \sqrt{\frac{|X'|}{|X|}}, & \text{if } x \in X \\ -\sqrt{\frac{|X|}{|X'|}}, & \text{if } x \in X' \end{cases}$, 前面取 ± 1 属于 $|X| = |X'|$ 的特殊情况

– 由前面的证明可知 $\sum_{(i,j) \in E} (x_i - x_j)^2 = \frac{1}{2} \sum_{i,j=1} A_{ij} (x_i - x_j)^2$

$$\Rightarrow \frac{1}{2} \sum_{i \in X, j \in X'} A_{ij} \left(\sqrt{\frac{|X'|}{|X|}} + \sqrt{\frac{|X|}{|X'|}} \right)^2 + \frac{1}{2} \sum_{i \in X', j \in X} A_{ij} \left(-\sqrt{\frac{|X|}{|X'|}} - \sqrt{\frac{|X'|}{|X|}} \right)^2$$

$$\Rightarrow \left(\frac{|X'|}{|X|} + 2 + \frac{|X|}{|X'|} \right) \frac{1}{2} \left[\sum_{i \in X, j \in X'} A_{ij} + \sum_{i \in X', j \in X} A_{ij} \right]$$

■ 谱聚类算法

□ 证明续

– 其中 $\frac{1}{2} \left[\sum_{i \in X, j \in X'} A_{ij} + \sum_{i \in X', j \in X} A_{ij} \right] = \text{cut}(X, X')$ 是割代价原始定义

– 因此 $\left(\frac{|X'|}{|X|} + 2 + \frac{|X|}{|X'|} \right) \frac{1}{2} \left[\sum_{i \in X, j \in X'} A_{ij} + \sum_{i \in X', j \in X} A_{ij} \right]$

$$\Rightarrow \text{cut}(X, X') \left(\frac{|X| + |X'|}{|X|} + \frac{|X| + |X'|}{|X'|} \right)$$

$$\Rightarrow (|X| + |X'|) \left(\frac{\text{cut}(X, X')}{|X|} + \frac{\text{cut}(X, X')}{|X'|} \right)$$

$$\Rightarrow (|V|)^2 \text{ratio cut}(X, X')$$

而 $(|X| + |X'|) = |V|$ 是一个常数

■ 谱聚类算法

□ 矩阵Q是对称的且是半正定矩阵

- $\forall x, x^T Q x = \sum x_i Q_{ij} x_j \geq 0$
- 矩阵Q的所有特征值均大于等于0

□ 矩阵Q最小特征值是0

- 且对应的特征向量为全1向量

□ 根据 Courant-Fischer minimax 定理，第2小的特征值是最小划分的解：

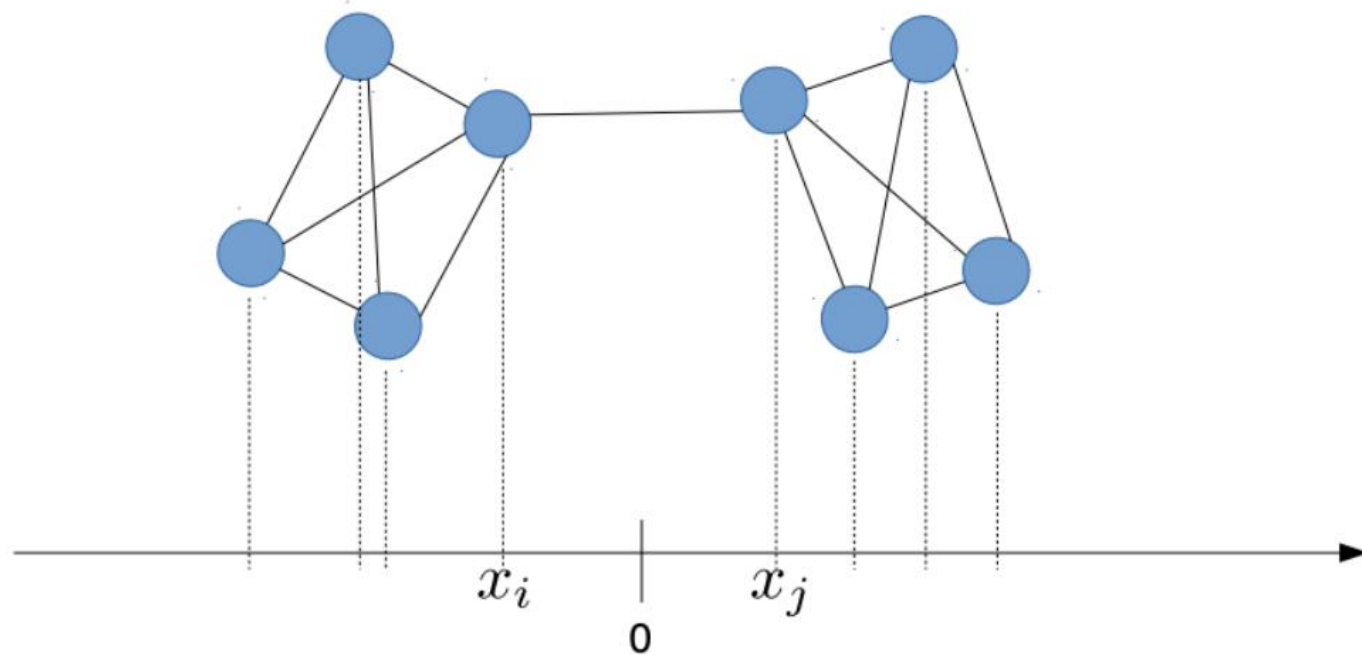
- 下面方程的解就是要找的最优划分

$$\lambda_2 = \min_{x \in S} \frac{x^T Q x}{|x|^2}$$

■ 谱聚类算法

□ 求解以下方程得到最优划分

$$\lambda_2 = \min_{x: \sum x_i = 0} \sum_{(i,j) \in E} (x_i - x_j)^2$$



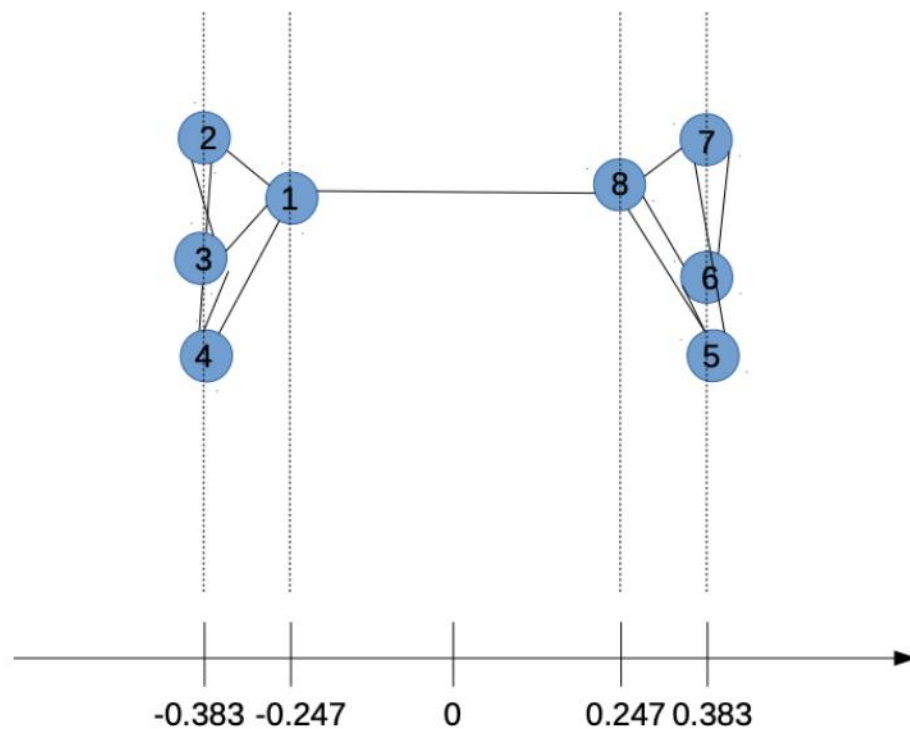
■ 谱聚类算法

□ 求解以下方程得到最优划分

– 下图中最优划分，根据正负号来判断

➤ $\{1,2,3,4\}$

➤ $\{5,6,7,8\}$



$$\lambda_1 = 0$$

$$\lambda_2 = 0.354$$

$$v_2 = \begin{bmatrix} 0.247 \\ 0.383 \\ 0.383 \\ 0.383 \\ -0.383 \\ -0.383 \\ -0.383 \\ -0.247 \end{bmatrix}$$

■ 谱聚类算法

□ 为什么是第2小的特征值

– 拉普拉斯谱矩阵最小特征值是0，不能选

- 对应一个全1的特征向量，而全1的特征向量在划分上的含义是所有的顶点属于一个划分，属于一个划分就没有割代价

– Courant-Fischer minimax 定理

- 对于任意 k 维子空间 A 中任意非0向量 y 的函数 $f(y)$ 的最小值存在上界，且上界为 $\lambda_k(A)$ ， $\lambda_k(A)$ 表示矩阵 A 的第 k 大特征值
- 即 $\lambda_k(A) = \max_{\dim(S)=k} \min_{|y|=1, y \in S} y^T A y$
- 想要 $y^T A y$ 最小，能直接知道的最小特征值是第二小特征值
 - 注意： $\lambda_k(A)$ 可能不是 $y^T A y$ 的全局最小值

■ 谱聚类算法

□ 算法的结果

- 最小二分割代价 $C(X, X') \geq n\lambda/4$
- 最小比例划分割代价 $C(X, X')/(|X||X'|) \geq \lambda/n$
- 其中的 λ 用矩阵第二小的特征值代入
 - 求解方程可以得到对应的划分方案


$$\lambda = \min_{x \in S} \frac{x^T Q x}{|x|^2}$$

注意 $|x|^2 = n$

□ 算法需求

- 只需要求一个特征向量
- 矩阵Q是一个对称的, 稀疏的矩阵, 容易求解

■ 谱聚类算法

□ 谱聚类应用

– 布局(placement)和布图规划(floorplan)

- [\[Hall 1970\]](#)
- [\[Otten 1982\]](#)
- [\[Frankle-Karp 1986\]](#)
- [\[Tsay-Kuh 1986\]](#)

– 二分下界与计算

- [\[Donath-Hoffman 1973\]](#)
- [\[Barnes 1982\]](#)
- [\[Boppana 1987\]](#)

– 比例划分下界与计算

- [\[Hagen-Kahng 1991\]](#)
- [\[Cong-Hagen-Kahng 1992\]](#)

Zhuo Feng

Associate Professor

[Department of Electrical and Computer Engineering](#)
[Michigan Technological University, Houghton, MI](#)

Biography

Zhuo Feng received the Ph.D. degree in [Electrical and Computer Engineering](#) from [Texas A&M University](#), College Station, TX, in 2005 and the B.Eng. degree in Information Engineering from [Xi'an Jiaotong University](#), Xi'an, China, in 2000. He is currently an Associate Professor in the [Department of Electrical and Computer Engineering, Michigan Technological University](#), Houghton, MI, where he is affiliated with the Computer Engineering Research Center. He has received the National Science Foundation (NSF) in 2014, a Best Paper Award from ACM/IEEE Design Automation Conference (DAC) in 2006 and 2008. He has served on the technical program committees of major international conferences including DAC, ISQED, and VLSI-DAT, and has been a technical referee for many leading IEEE/ACM journals in VLSI and parallel computing. He is a Senior Member of IEEE. In 2016, he became a co-founder of [LeapLinear Solutions](#) to develop high-performance spectral methods for numerical and graph problems (networks) with billions of elements, based on the latest breakthroughs in spectral graph theory.

Research

- [High-performance spectral methods for numerical and graph problems](#)
 1. Spectral sparsification of undirected graphs ([DAC'16](#), [DAC'18](#), [software](#)) and directed graphs ([arXiv:1812.04165](#))
 2. Sparsified algebraic multigrid (SAMG) for solving SDD matrices ([ICCAD'17](#))
 3. Spectral graph reduction for scalable graph partitioning and data visualization ([arXiv: 1812.08942](#), [DAC'19](#))
 4. Spectral methods for data clustering and network reduction ([arXiv:1710.04584](#), [ICCAD'14](#))

■ 网络流算法

- 最小割平衡划分算法^[1]
 - 基于最大流最小割理论提出的划分算法
- 基于逻辑门复制的划分算法^[2]
- 带空间和引脚约束的多路划分算法^[3]
- 多资源约束划分算法^[4]
- 时分多路复用FPGA划分算法^[5]

在2003年被选为
ICCAD近20年来最
伟大的工作之一

[1] Hannah Honghua Yang, D.F. Wong, "Efficient network flow based min-cut balanced partitioning," ICCAD, 1994.

[2] Hannah Honghua Yang, D.F. Wong, "New algorithms for min-cut replication in partitioned circuits," ICCAD, 1995.

[3] Huiqun Liu, D.F. Wong, "Network flow based multi-way partitioning with area and pin constraints," ISPD, 1997.

[4] Huiqun Liu, Kai Zhu, D.F. Wong, "Circuit Partitioning with Complex Resource Constraints in FPGAs," FPGA, 1998.

[5] Huiqun Liu, D.F. Wong, "Network flow based circuit partitioning for time-multiplexed FPGAs," ICCAD, 1998.

■ 网络流算法

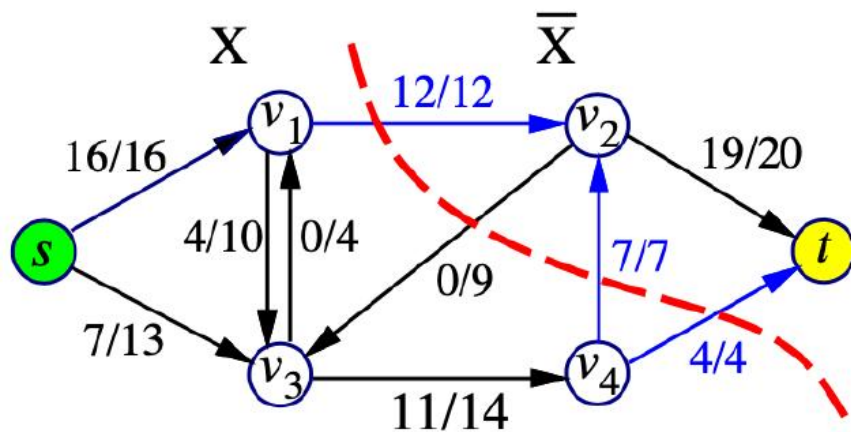
□ 流(flow)的定义

- 网络流 $G = (V, E)$ 是有向图, 其中任意边 $(u, v) \in E$ 有容量限制 $c(u, v) > 0$
- 存在一个节点没有输入边(输出边), 称之为源点 s (汇点 t)
- 一个流 $f: V \times V \rightarrow R$ 满足
 - 容量限制(Capacity constraint): $f(u, v) \leq c(u, v), \forall u, v \in V$
 - 反对称(Skew symmetry): $f(u, v) = -f(v, u), \forall u, v \in V$
 - 流量守恒(Flow conservation): $\sum_{v \in V} f(u, v) = \sum_{v \in V} f(v, u), \forall u \in V - \{s, t\}$
- 一个流的价值 $f: |f| = \sum_{v \in V} f(s, v) = \sum_{v \in V} f(v, t)$
 - 即能够提供从 s 到 t 顶点的容量
 - 举个例子, 自来水厂到你家的管网 G 十分复杂。管网 G 的最大流就是通过管网 G , 自来水厂能向你家输送的最大水流量

■ 网络流算法

□ 最大流问题

- 给定一个带有源点 s 和汇点 t 的流网络图 G ，找到从 s 到 t 的一个流，它的价值最大
- 例如下图中流的最大价值(容量)为 $12 + 7 + 4 = 23$



■ 网络流算法

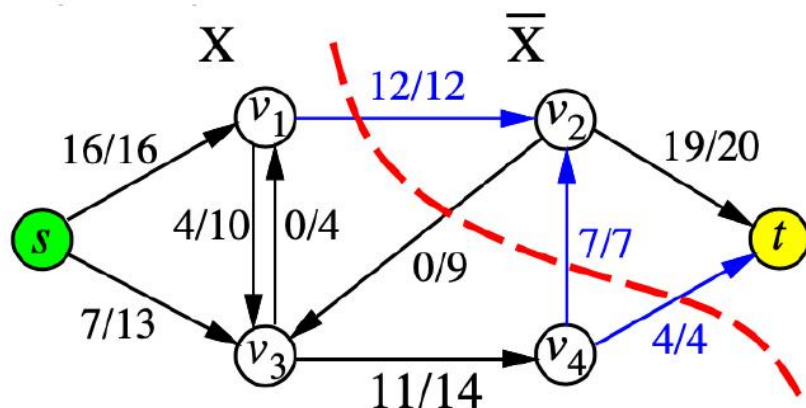
□ 最大流与最小割

- 对于一个流网络 $G = (V, E)$ 的划分 $(X, \bar{X})$, 是指存在一个划分满足 $X + \bar{X} = V$ 且 $s \in X$ 和 $t \in \bar{X}$

➤ 一个划分的容量: $cap(X, \bar{X}) = \sum_{u \in X, v \in \bar{X}} c(u, v)$ (仅计算 X 指向 $\bar{X}$ 的边)

- 最大流最小割理论^[1]

➤ f 是一个最大流, 在一些情况下 $|f| = cap(X, \bar{X})$, 对于划分 $(X, \bar{X})$ 来说就是最小割的割代
价



此时最大流 $|f| = 16 + 7 = 23$
最小割 $cap(X, \bar{X}) = 12 + 4 + 7 = 23$

[1] L.R. Ford Jr, D.R. Fulkerson, "Maximal Flow Through a Network," Canadian journal of Mathematics, vol 8, pp. 399-404, 1956.

■ 网络流算法

□ 定义

- **增广路径** p 表示一条满足以下要求的从 s 到 t 的简单路径
 - 同向边: $(u, v) \in E$ 和 $s \rightarrow t$ 的路径方向一致
 - 反向边: $(u, v) \in E$ 和 $s \rightarrow t$ 的路径方向相反
 - 对于路径 p 上的每一条同向边 (u, v) , 流不超过边的容量 $f(u, v) < c(u, v)$
 - 对于路径 p 上的每一条反向边 (u, v) , 满足 $f(u, v) > 0$
- f 是最大流 $\Leftrightarrow$ 不存在任何增广路径 p
 - **算法搜索增广路径直到增广路径不存在为止**, 最后把属于最大流合适的边切开得到划分
 - 最早由Ford & Fulkerson在1956年提出最大流算法^[1], 时间复杂度为 $O(|E||f|)$
 - 第一个多项式时间复杂度 $O(|E|^2|V|)$ 算法由Edmonds & Karp^[2]在1972年提出
 - Goldberg & Tarjan^[3]在1986年降低时间复杂度到 $O(|E||V|\log(|V|^2/|E|))$

[1]L.R. Ford Jr, D.R. Fulkerson, "Maximal Flow Through a Network," Canadian journal of Mathematics, 1956.

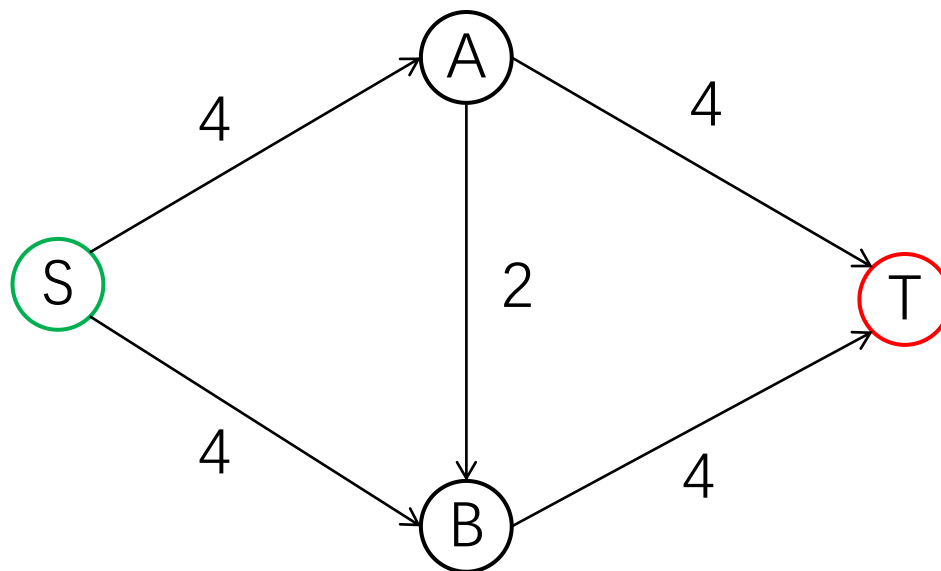
[2]Jack Edmonds, Richard M. Karp, "Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems," Journal of ACM, 1972.

[3]Andrew V. Goldberg, Robert E. Tarjan, "A new approach to the maximum-flow problem," STOC, 1986.

■ 网络流算法

□ Ford & Fulkerson的最大流算法示例

– 搜索从 s 到 t 顶点的最大流

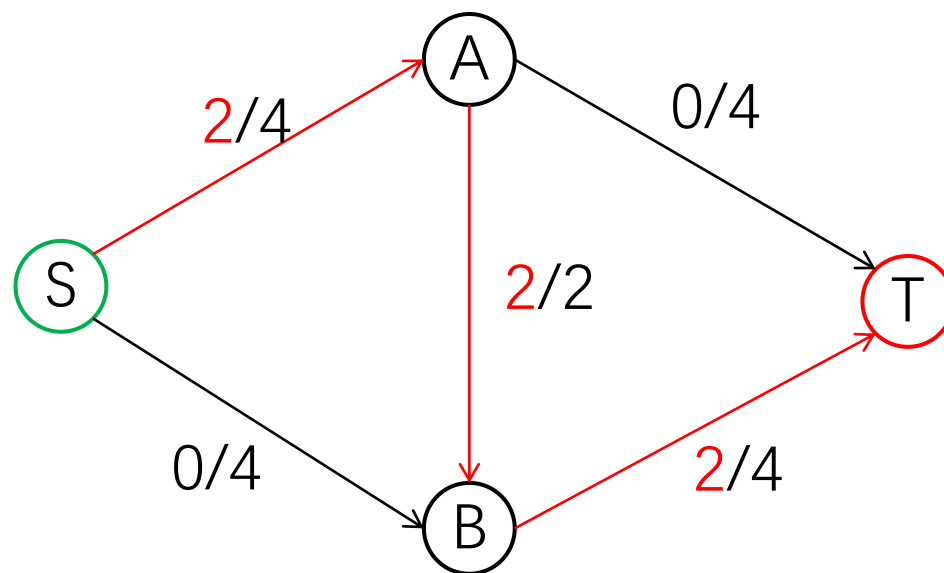


■ 网络流算法

□ 第一次增广

– 增广路径为(S,A,B,T)

➤ 仅考虑流不超过边的容量，这条增广路的容量为2



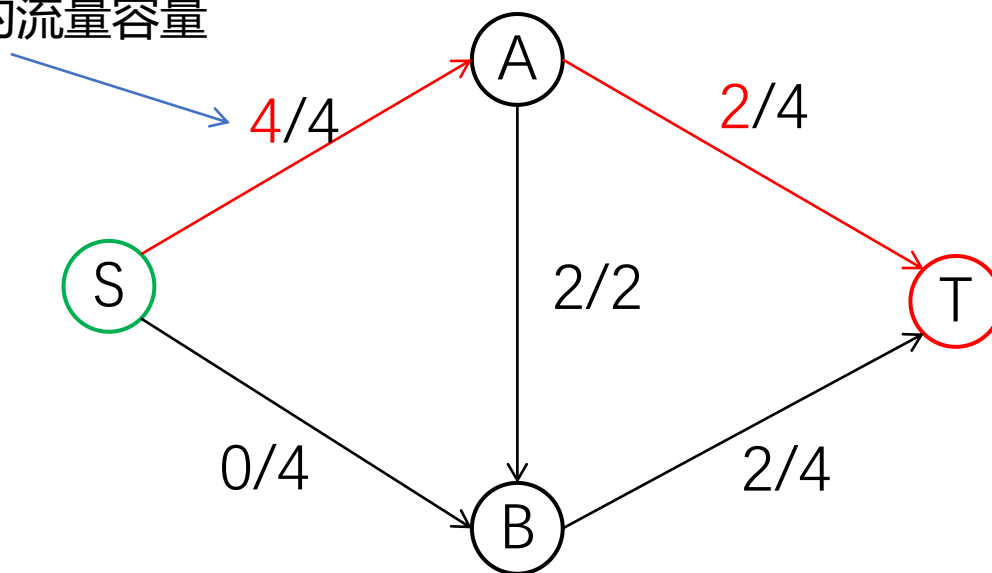
■ 网络流算法

□ 第二次增广

– 增广路径为(S,A,T)

➤ 增广路的容量为2

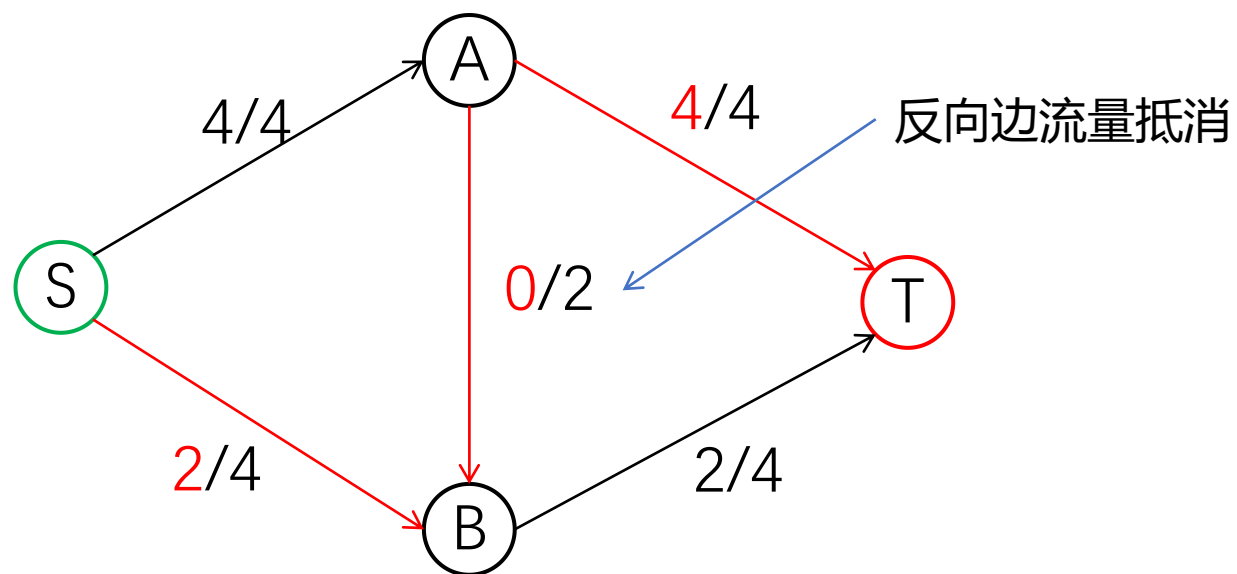
仅剩下2的流量容量



■ 网络流算法

□ 第三次增广

- 增广路径为(S,B,A,T)
 - 此时A到B的边是反向边

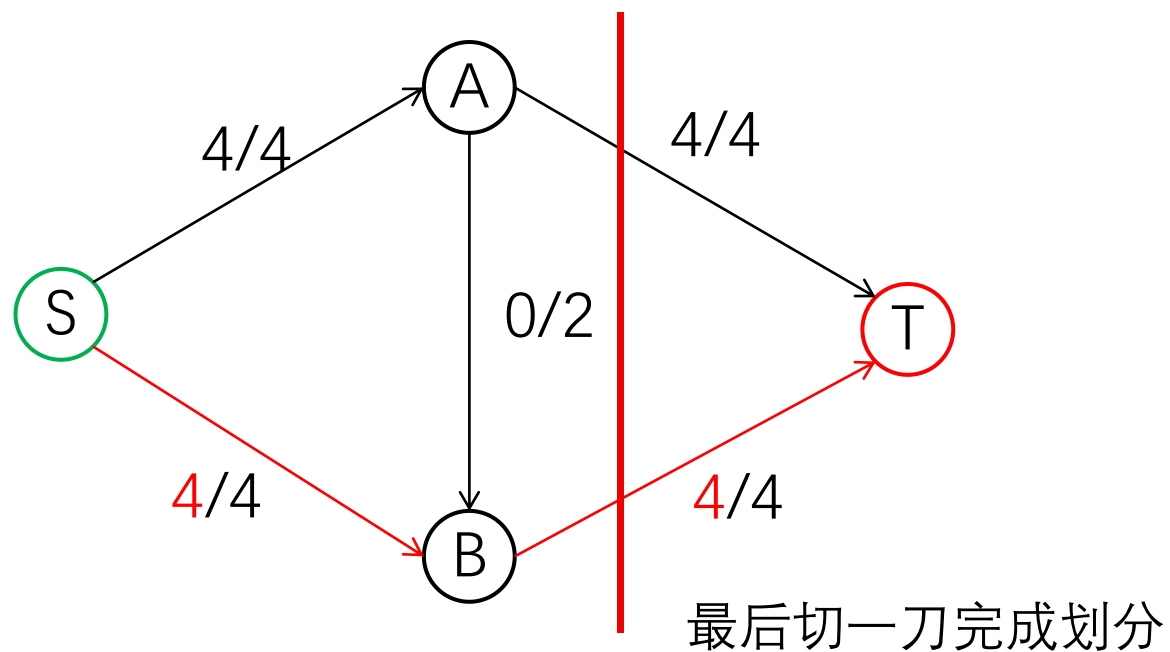


■ 网络流算法

□ 第四次增广

– 增广路径为(S,B,T)

➤ 不存在新的增广路径p，已经找到最大流，且最大流为8



■ 网络流算法

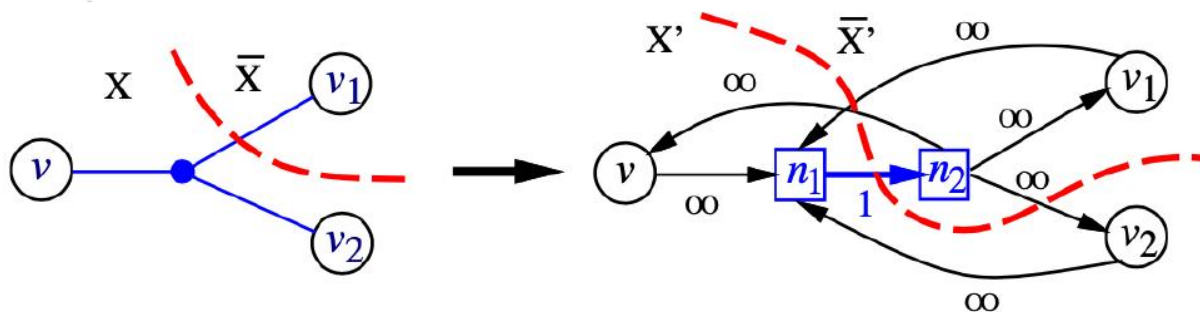
□ 基于网络流的划分算法

- 但这项技术并没有广泛应用于划分
 - 网络流算法仅适用于图，而不是超图
 - 可能会导致非平衡的划分结果，且算法效率低(调用多次最大流算法进行划分)
- Yang & Wong 提出网络流的平衡流划分算法FBB(Flow-Balanced-Bipartition)解决上面的问题，做出以下的改进
 - 提取网表超图用网络流进行建模
 - 最小二分(非平衡)优化算法
 - 划分算法的高效实现：与一次最大流算法时间开销渐近

■ 网络流算法

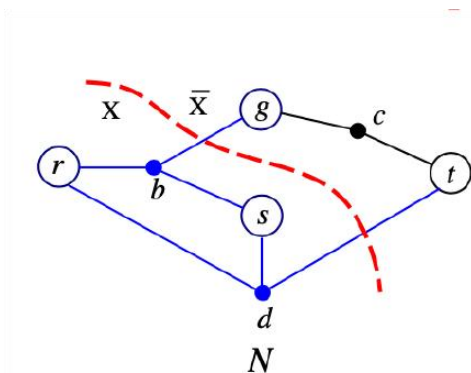
□ FBB算法初始化超图

- 把超图 $N = (V, E)$ 按照以下顺序建模为一个网络流可求解的图 $N' = (V', E')$
 - 对于每一条超边连接的顶点集合 $n = \{v: v_1, \dots, v_l\}$, 新增两个新顶点 n_1, n_2 和一条**桥边** $(n_1, n_2) \in E'$
 - 对于集合 $n = \{v: v_1, \dots, v_l\}$ 中的每一个顶点 v , 新增两条边 (v, n_1) 和 (n_2, v)
 - 令 s 成为图 N' 的源点, t 成为图 N' 的汇点
 - 所有的桥边容量设置为1, 其余边的容量设置为 ∞
 - V' 中顶点原属于 V 的, 顶点权重继承。其余顶点权重为0



■ 网络流算法

□ 例如



超图 N
顶点集 $V = \{g, r, s, t\}$
超边 E
 $\{r: g, s\}$
 $\{s: r, t\}$
 $\{g: t\}$

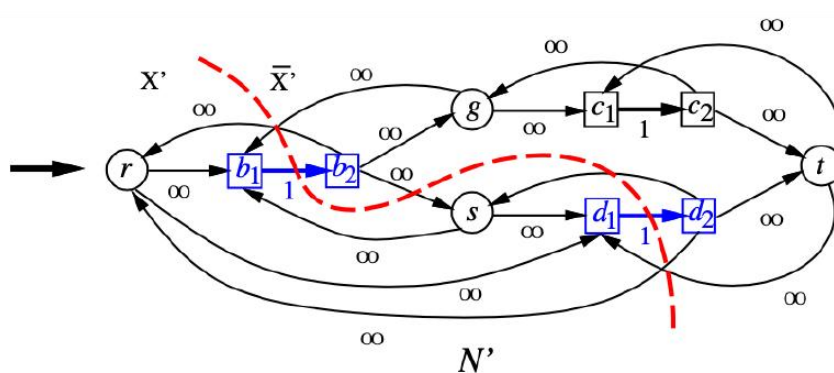


图 N'
顶点集 $V = \{b_1, b_2, c_1, c_2, d_1, d_2, g, r, s, t\}$
边 E
 $\{r: b_1, b_2, d_1, d_2\}$
...

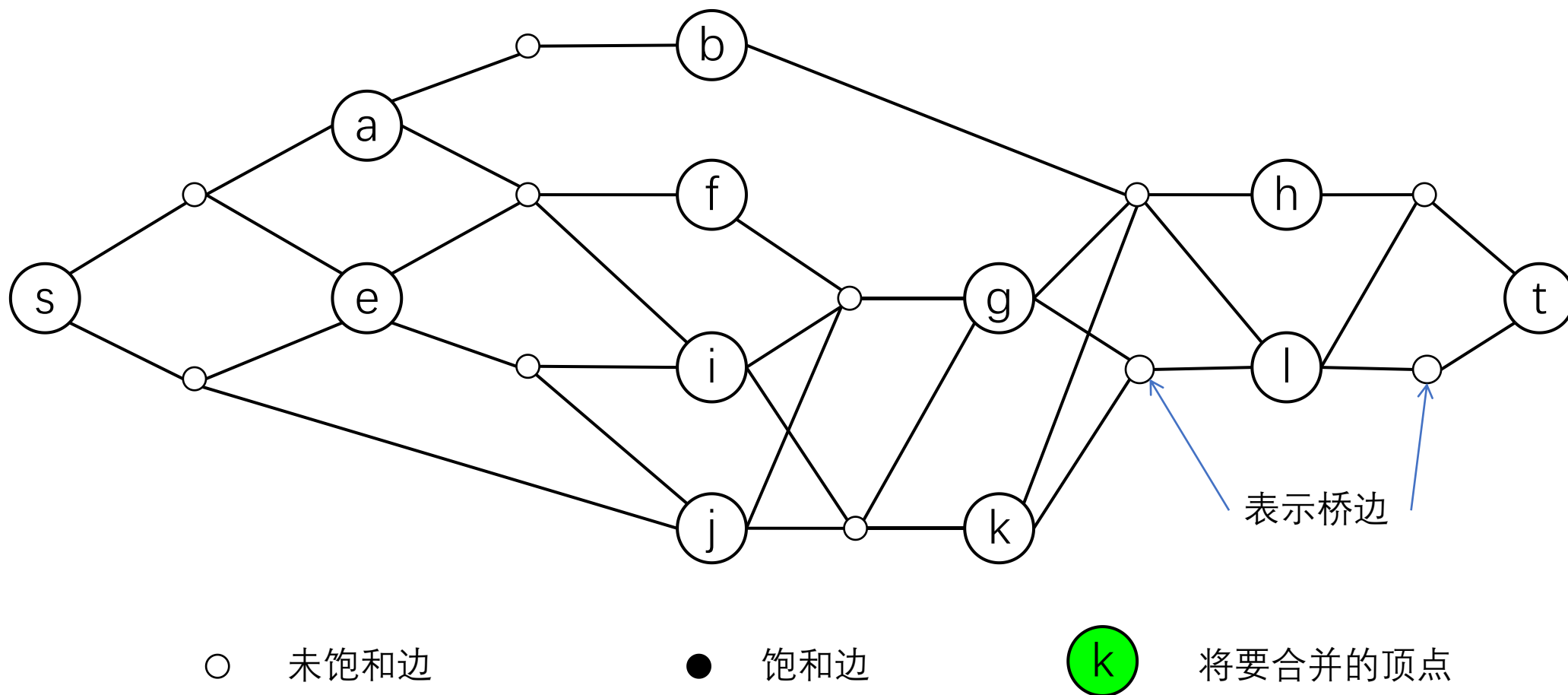
■ 网络流算法

□ FBB算法流程

- 0.从图 N' 上随机选择一对点作为源点 s 和汇点 t
- 1.基于图 N' 计算最大流, 然后划分成 (X, X') , 假设 X 划分包含源点 s
- 2.如果满足 $(1 - \varepsilon)rW \leq w(X) \leq (1 + \varepsilon)rW$, 那么返回结果
 - W 表示图 N' 全部顶点权重和。 r 是一个参数, 控制划分均衡性。 ε 是一个超参数
- 3.如果 $w(X) < (1 - \varepsilon)rW$
 - 把 X 的所有顶点合并到源点 s
 - 在划分边界附近靠近 t 侧的顶点按一定概率合并到源点 s
 - goto 1
- 4.如果 $w(X) > (1 + \varepsilon)rW$
 -

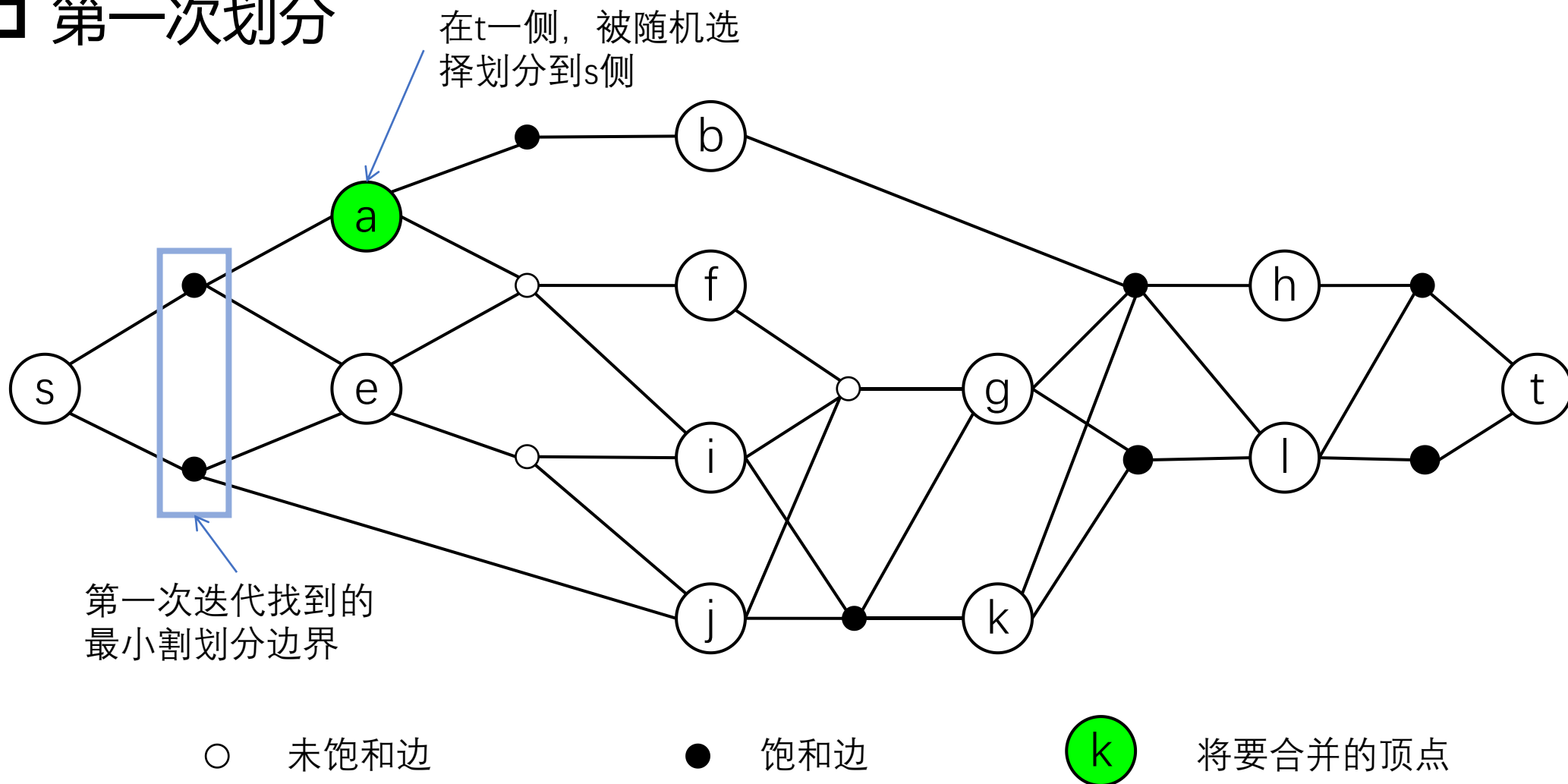
■ 网络流算法

□ 举例



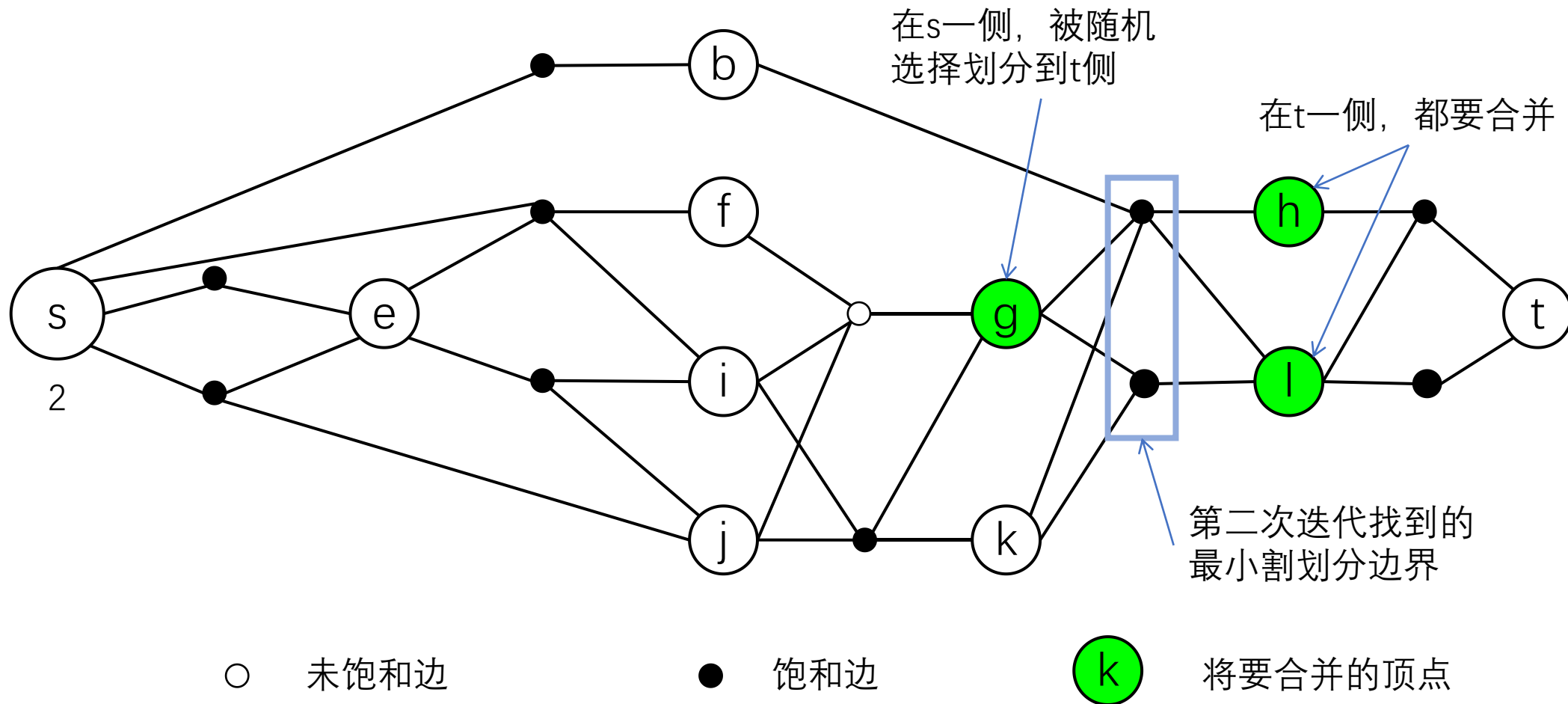
■ 网络流算法

□ 第一次划分



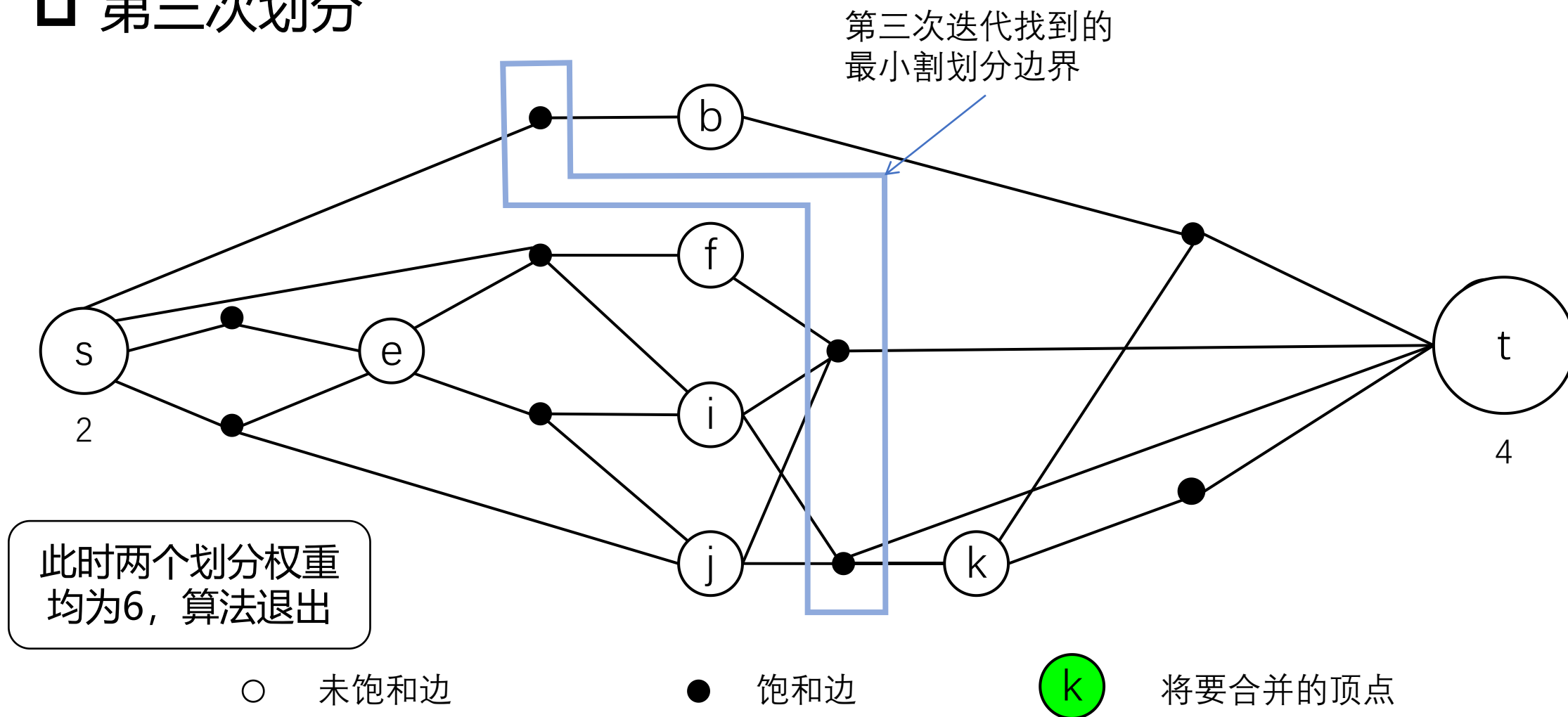
■ 网络流算法

□ 第二次划分



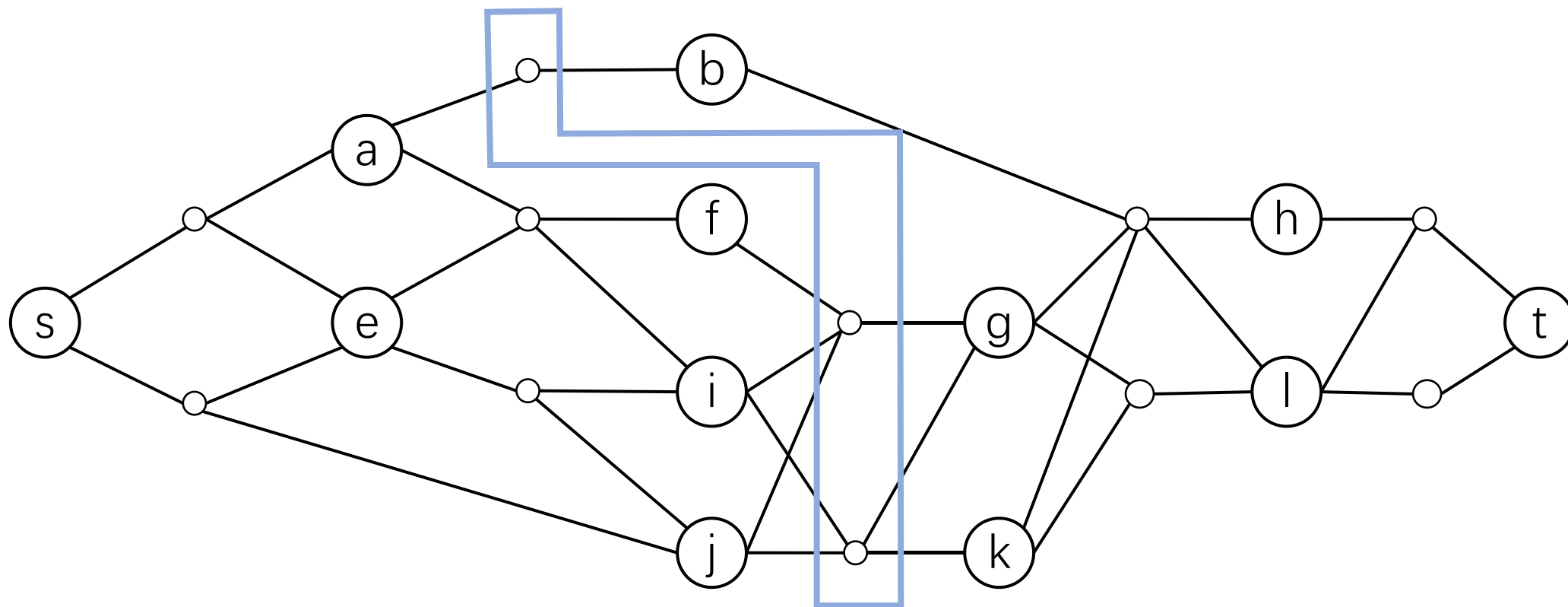
■ 网络流算法

□ 第三次划分



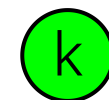
■ 网络流算法

□ 找到最优划分



○ 未饱和边

● 饱和边



将要合并的顶点

■ 网络流算法

□ 增量流更新

- 重复的求解最大流问题可能非常耗时
- 实际上不需要每次迭代都重新计算一遍最大流
- 在上一次迭代的最大流基础上进行更新，仍能保证算法的正确性

□ FBB算法的时间复杂度 $O(|V||E|)$ ，和运行一次最大流算法一致

- 最多需要 $|V|$ 次增广路径的求解，增广路的求解复杂度 $|E|$
 - 每次增广路径的求解，无论增广路径，还是新的最大流被找到，至少有一个顶点会被合并到源点 s 或汇点 t ，所以最多只会运行 $|V|$ 次
 - 最差情况下增广路搜索算法要搜完所有边才能确定是否存在
 - 只是找增广路，所以只需要找到即可，无关这条路是否最小、最优。因此只需要遍历一遍所有边，复杂度为 $|E|$

■ 总结

□ 算法总结

– 贪心算法

- Kernighan-Lin 算法
- Fiduccia-Mattheyses 算法
- 多层次划分算法: hMETIS

快速&灵活, 但是
解质量不稳定

– 搜索算法

- 模拟退火

– 分析算法

- 谱聚类
- 最大流

理论上合理, 但效
率不高

感谢！
